Ders 2 Çalışma Özeti

Ders 2 Çalışma Özeti

Genel Konular

- Kısa tekrar ve dersin akışı
  - Tasarım kalıpları, nesneye dayalı modellemede sık karşılaşılan sorunlar için önerilen iyi çözümlerdir; işin nasıl doğru ve iyi yapılacağını anlatır. Dersin büyük kısmı GoF kalıplarına, son kısmı code smells ve refactoring konusuna ayrılır.
  - Kalıplar GoF kitabındaki sırayla anlatılır: kitap önce MVC'yi verir, ardından yaratımsal (creational) kalıplarla devam eder. Kümeler: yaratımsal, yapısal (structural), davranışsal (behavioral).
  - Kalıp adları Türkçeye çevrilmez; literatürde yerleşmiş orijinal adlarıyla öğrenilir.
- Abstract Factory: amaç, örnek, sorun (ekrandan, 00:16:50)
  - Amaç: birbiriyle ilişkili ya da aynı aileden olan nesneleri, sınıflarını belirtmeden oluşturabilmek.
  - Normalde nesne oluşturmak için sınıf adını taşıyan kurucu (constructor) çağrılır; bunu sınıf adı vermeden yapmayı kalıtım ve çok biçimlilik mümkün kılar.
  - Örnek: Java'da GUI bileşenlerinin görünüşünün (look-and-feel) çalışma anında değiştirilebilmesi. GoF kitabı yazıldığında Java yoktu; kitaptaki örnek farklı pencere yöneticileridir (Motif, Presentation Manager; güncel karşılıkları GNOME, KDE).
  - Sorun: taşınabilirliği sağlamak için istekçinin (client) hangi tür görünüşün hangi sınıftan nesnelerle gerçeklendiğini bilmemesi gerekir. İstekçi yalnız "bana bir pencere, bir kaydırma çubuğu çiz" diyebilmelidir.
- Örnek çözüm: WidgetFactory (ekrandan, 00:20:00 ve 00:39:30)
  - Client, soyut `WidgetFactory` ile konuşur; `createScrollBar()` ve `createWindow()` soyut metotlarını çağırır. Widget, her GUI bileşenine (düğme, pencere, kaydırma çubuğu, radyo düğmesi) verilen genel addır.
  - Somut alt sınıflar `MotifWidgetFactory` ve `PMWidgetFactory`; biri `MotifScrollBar`/`MotifWindow`, öteki `PMScrollBar`/`PMWindow` döndürür. Client yalnız soyut `Window` ve `ScrollBar` türlerini görür (yerine geçirebilirlik ilkesi).
  - Hoca GoF şemasını güncel UML'e çevirerek yeniden çizdi: soyut sınıflar «`abstract`» stereotipiyle, soyut metotlar `{abstract}` ile, dönüş türleri yazılarak, kalıtım içi boş üçgenle; Client ve somut fabrikalardan ürünlere giden ilişkiler kesikli (kullanma) oklarla.
- Switch-case seçeneği ve zayıf yönü (ekrandan, 00:40:00)
  - Slayt: `WidgetFactory.setLoookAndFeel( val: EnumLookFeel )`, `switch( val ) { … }`. Zayıf yönü: yeni görünüş biçimleri için yeni enum değerleri ve switch içine yeni case komutları eklemek gerekir.
  - GNOME ve KDE eklenince her case'e 20-30 satırlık yeni kod bloğu girer; metot uzar (ileride görülecek "Long Method" kod kusuru), okunabilirlik ve bakım kolaylığı düşer.
- Kalıp yapısı ve bileşenleri (ekrandan, 00:45:00)

```
Client ──> AbstractFactory {CreateProductA(), CreateProductB()}
               △── ConcreteFactory1 ··> ProductA1, ProductB1
               △── ConcreteFactory2 ··> ProductA2, ProductB2
```

```
Client ──> AbstractProductA  △── ProductA1, ProductA2
Client ──> AbstractProductB  △── ProductB1, ProductB2
```

  - ProductXY: oluşturulacak nesneler (ürünler). X hangi ürün olduğunu (A, B: örneğin pencere, kaydırma çubuğu), Y hangi aileden olduğunu (1, 2: örneğin Motif, PM) söyler.
  - AbstractProductX: X ürünlerinin arayüzlerini tanımlar. ConcreteFactoryY: Y ailesinin ürünlerini oluşturur. AbstractFactory: ürün oluşturmak için gereken soyut metotların imzalarını içerir.
  - Adında "factory" geçen kalıp, sınıf ya da metot nesne oluşturmakla görevlidir.

- Kalıbın uygulanabileceği durumlar (ekrandan, 00:50:00)
  - Sistem, kendi ürünlerinin nasıl oluşturulduğundan, birleştirildiğinden ve sunulduğundan bağımsız olmalıysa.
  - Sistem, farklı ailelerden herhangi birinin nesneleriyle yapılandırılmalıysa.
  - İlişkili ürün ailelerinin nesneleri birlikte kullanılmalıysa.
  - Bir ürün ailesi sınıf kütüphanesi (framework) olarak sunulacak, gerçeklemeler yerine yalnız arayüzler açığa çıkarılacaksa. Facade kalıbı da benzer işe yarar ama daha karmaşıktır; bu tür sorunlarda Abstract Factory tercih edilir.
- Uygulama örneği: 3B oyun motoru (01:06:14)
  - Oyun motoru farklı işletim sistemi ve ekran kartlarında çalışmak için OpenGL ya da DirectX'in (10, 11, 12 sürümleri) bileşenlerini kullanmalı: Renderer (3B görüntüyü bakış açısına göre ekrana çizme) ve Shader (gölgelendirme).
  - Hoca her örnekte önce akla ilk gelen (belki yanlış) çözümü, sonra bir değişiklik isteğini, sonra kalıplı çözümü gösterir; değişikliğin hangi tasarımda ne kadar zahmetli olduğu karşılaştırılır. Örnek kodlar derlenebilen iskelet kodlardır; sınavlarda kod değil tasarım istenir.
- Çözüm 1: switch-case (ekrandan, 01:10:00 ve 01:15:00)
  - `ApControl` (Application Control, oyunun ana modülü) `LibraryType` enum'u ile `OpenGLRenderer or`, `OpenGLShader os`, `DirectXRenderer dr`, `DirectXShader ds` üye alanlarını tutar; `doRendering()` ve `doShading()` içinde `switch (library)` ile ilgili nesnenin `renderOpA/B/C()` ya da `shadeOpA/B/C()` metotlarını çağırır.
  - Sınıf şemasında `ApControl` dört sınıfla bire bir sahiplik (düz ok) ilişkisindedir.
  - DirectX 12 eklemek: iki yeni sınıf (`DirectXRenderer12`, `DirectXShader12`), enum'a yeni değer, `ApControl`'e yeni üye alanlar ve her `do...` metoduna yeni case. Texture mapping gibi yeni bir yetenek eklenirse her kütüphane için tekrar edilir.
- Çözüm 2: kalıtımın doğru kullanımı (ekrandan, 01:27:45)
  - İlke: değişeni bul ve kalıtım hiyerarşisine koy. Soyut `Shader` ve `Renderer` sınıfları, altlarında `OpenGLShader`/`DirectXShader` ve `OpenGLRenderer`/`DirectXRenderer`.
  - `ApControl` yalnız `Renderer r` ve `Shader s` tutar; switch-case yalnız kurucuda (nesneleri ilklendirirken) kalır; `doRendering()` yalnız `r.renderOpA(); r.renderOpB(); r.renderOpC();` çağırır.
  - DirectX 12 eklemek: iki kardeş sınıf (`DirectX12Renderer`, `DirectX12Shader`) ve kurucuda bir case.
- Yanlış kurulan kalıtım/sahiplik (ekrandan, 01:40:00, 01:50:00 ve 02:39:20)
  - Kütüphane başına sınıf (`OpenGL`, `DirectX`, `DirectX12`), her biri kendi Renderer ve Shader'ına sahip ve `shadeOpA()` gibi çağrıları onlara ileten delegate metotlar içerir (`public void shadeOpA() { s.shadeOpA(); //delegation }`). Araya soyut bir `GameEngine` sınıfı konsa bile DirectX 12 için üç sınıf eklemek gerekir; hoca bunu "tercih edilmeyecek modelleme" diye işaretledi.
  - Bir başka kötü seçenek: oyunun tamamını türetmek (`«Abstract» APControl` altında `OpenGL_APControl` ve `DirectX_APControl`). Oyun mantığındaki bir hata her alt sınıfta ayrı ayrı düzeltilmek zorunda kalır (kod tekrarı).
- Çözüm 3: Abstract Factory (ekrandan, 02:05:45 ve 02:07:40)

  - `ApControl` client'tır. Soyut `ComponentFactory`'nin `createRenderer(): Renderer` ve `createShader(): Shader` soyut metotları vardır; `OpenGLFactory` ve `DirectXFactory` bunları ilgili somut ürünü döndürecek biçimde gerçekler.
  - Kurucuda `r = library.createRenderer(); s = library.createShader();`. Hangi fabrikanın kullanılacağı yine bir yerde seçilmelidir: (A) kurucuda switch-case, (B) fabrikayı kurucuya parametre olarak vermek (`ApControl(ComponentFactory library)`), (C) her kütüphane için ayrı dağıtım (ya da yapılandırma programının yazdığı bir yapılandırma dosyasından okumak). Hoca B'yi tercih etti.
  - DirectX 12 eklemek: `DirectX12Factory` ile birlikte iki ürün sınıfı, yani Çözüm 2'den bir sınıf fazla. Bu örnekte kalıbın, doğru kurulmuş kalıtıma göre görünür bir avantajı yoktur.
- Preserve Whole Object ile son hal (ekrandan, 02:31:40)
  - Slayt: refactoring konusundan sonra Preserve Whole Object eyleminin yürütülmüş hali tercih edilir. `ApControl` yalnız `ComponentFactory factory` tutar, kurucuda `factory = aFactory;`, metotlarda `factory.r.renderOpA();` gibi çağrılar.
  - Kazanç: Texture Mapper gibi yeni bir yetenek geldiğinde `ApControl`'e yeni üye alan ve ilklendirme satırı eklemek gerekmez.

Hocanın Özellikle Vurguladığı Kısımlar

- Switch-case bir uyarı işaretidir
  - Nesneye dayalı kod yazarken kendinizi switch-case kullanırken görürseniz büyük olasılıkla doğru bir kalıtım ilişkisi ve çok biçimlilik eksiktir. Daha küçük bir olasılıkla da sorumluluklar yanlış sınıflara/metotlara atanmıştır.
- Switch-case tamamen yok olmaz
  - Kalıp kullansak bile switch-case "öldürülemez"; yalnız ilklendirme noktasına kadar geri çekilir. Bu iş kurucuda, bir yapılandırma programında ya da dağıtım aşamasında yapılabilir.
- Güncel UML gösterimi kullanılmalı
  - GoF kitabının şemaları UML standartlaşmadan önce çizildiği için kalıtım oku farklıdır (hocaya göre ER şemalarındaki karga ayağını andırır). Hoca bu şemaları güncellemedi ama öğrencilerden güncel UML gösterimini istiyor.
  - İtalik yazı elle çizimde ayırt edilemez; soyut sınıflar için «`abstract`» stereotipi, soyut metotlar için `{abstract}` kullanılmalı. Bire bir sahiplikte nicelik yazılmayabilir, bire çok sahiplikte * konur. Şema ne kadar ayrıntılıysa tasarımın doğru anlaşılıp anlaşılmadığı o kadar iyi görülür.
- Kalıp illa kullanılmak zorunda değil
  - Bu örnekte kalıtımı doğru kuran çözüm kalıplı çözüm kadar iyidir, hatta bir sınıf daha az ister. Ancak kalıtım hiyerarşisini kurmak deneyimsiz biri için risklidir; yanlış kurulursa kalıplı çözümden daha kötü bir yere varılır.
- Ölçüt satır sayısı değil, bakım çabasıdır
  - İki tasarımın satır sayısı yakın olabilir; önemli olan bakım ya da yenilik çalışmasında gereken çabayı artırmamaktır. Yazılım projelerinde çabanın yaklaşık yarısı bakım aşamasında harcanır.
- Kalıp adları orijinal İngilizce adlarıyla bilinmeli.

Kısa Tekrar Notları

- Abstract Factory: ilişkili nesne ailelerini somut sınıflarını belirtmeden oluşturur.
- Bileşenler: AbstractFactory (oluşturma metotlarının imzaları), ConcreteFactoryY (Y ailesini üretir), AbstractProductX (X ürününün arayüzü), ProductXY (somut ürün), Client.
- Client yalnız soyut fabrika ve soyut ürünlerle çalışır; somut aileyi bilmez (kara kutu).
- Adında "factory" geçen öğe nesne oluşturmakla görevlidir.
- Switch-case çözümü: her yeni aile için enum değeri ve her metoda yeni case; uzun metot kusuru.
- Kalıtımla çözüm: değişeni (Renderer, Shader) soyutla; switch yalnız kurucuda kalır.
- DirectX 12 eklemek: Çözüm 1'de 2 sınıf + enum + birçok case; Çözüm 2'de 2 kardeş sınıf + 1 case; Çözüm 3'te 1 fabrika + 2 ürün sınıfı.

- Delegate metotlarla dolu kütüphane sınıfları ve oyunun tamamını türetmek tercih edilmeyen modellemelerdir.
- Fabrika seçimi: kurucuda switch-case, kurucuya parametre ya da ayrı dağıtım/yapılandırma dosyası.
- Üç genel ilke: değişeni bul ve sarmala; kalıtım yerine parça-bütün ilişkisini tercih et; gerçeklemelere göre değil arayüzlere göre tasarla.
- Sonraki kalıp: Factory Method (adı benzese de farklı bir sorunu çözer).

Detaylı Açıklamalar

- UML çizim aracı ve gösterim ayrıntıları
  - Hoca şemaları Violet UML Editor ile çizer: çift tıklamayla açılan küçük bir jar dosyası (JRE ya da JDK gerekir); şemayı `.class.violet` dosyası olarak kaydeder ve PNG olarak dışa aktarır, kod üretme ya da koddan şema çıkarma yeteneği yoktur. Koddan şema üretmek için bir süre Eclipse'in eUML2 eklentisini kullanmış, ürettiği şemalar karmaşık olduğu için bırakmıştır.
  - Violet'te «`abstract`» stereotipi doğrudan verilemediği için hoca bir interface öğesi oluşturup "interface" yazısını "abstract" olarak değiştirir. Bütün metotları soyut olan ve üye alanı olmayan bir sınıf (WidgetFactory gibi) rahatlıkla interface olarak da kodlanabilir; GoF kitabı yazıldığında her dil interface'i desteklemiyordu.
  - Görünürlük: + public, - private. Üye alanlar veri gizleme ilkesi gereği her zaman private; metotlar genelde public, iç ayrıntıyı içerenler private. `{abstract}` gösterimindeki küme parantezleri metot gövdesi değildir; `{final}` ya da `{abstract, final}` gibi nitelikler de bu biçimde yazılır.
  - Kesikli ok kullanma (bağımlılık) ilişkisidir: örneğin `PMWidgetFactory`, `PMScrollBar` ve `PMWindow` döndüren metotlara sahip olduğu için bu sınıflarla kullanma ilişkisindedir (üye alan değil, dönüş değeri, parametre ya da geçici değişken).
- WidgetFactory şemasının güncel UML hali (ekrandan, 00:39:30)

```
«abstract» WidgetFactory
  + createScrollBar() : ScrollBar {abstract}
  + createWindow() : Window {abstract}
    △── MotifWidgetFactory
    │     + createScrollBar() : MotifScrollBar
    │     + createWindow() : MotifWindow
    △── PMWidgetFactory
          + createScrollBar() : PMScrollBar
          + createWindow() : PMWindow
«abstract» Window     △── MotifWindow, PMWindow
«abstract» ScrollBar  △── MotifScrollBar, PMScrollBar
Client ··> WidgetFactory, Window, ScrollBar
MotifWidgetFactory ··> MotifWindow, MotifScrollBar
PMWidgetFactory ··> PMWindow, PMScrollBar
```

  - Ders sonrası not: Dışa aktarılan PNG'de `Window` altındaki iki sınıf da yanlışlıkla `MotifScrollBar` ve `PMScrollBar` adını taşıyor (ekrandan, 00:39:30); hoca bunu fark edip Window olarak düzeltti (00:45:52). Doğrusu `MotifWindow` ve `PMWindow`'dur.
- Neden kalıtım ve çok biçimlilik?
  - Alt sınıftan bir nesne, üst sınıftan bir nesnenin yerine her zaman ve her bağlamda kullanılabilir. Client'a somut `MotifWidgetFactory` verilse de client onu yalnız `WidgetFactory` olarak görür; aldığı kaydırma çubuğu da arka planda `MotifScrollBar`'dır. Deneyimli tasarımcılar kalıtımın gücünün yanı sıra yanlış kullanımının tehlikelerinin de farkındadır.
- Çözüm 1'den Çözüm 2'ye geçişte ne kazanıldı?
  - Çözüm 1'de switch-case hem `doRendering` hem `doShading` içindedir ve her yeni işlem (`doNotation`, `doCollisionDetection` vb.) için çoğalır. Çözüm 2'de karmaşık render/shade komutları kaybolmaz, `DirectXRenderer` gibi somut sınıfların içine taşınır;

yeni bir kütüphane sürümü için uzun metotlara n tane case eklemek yerine kurucudaki kısa switch'e bir case eklemek yeter.

- Yanlış tasarımda neden delegate metotlar var?
  - Kütüphane sınıfı (`OpenGL`) Renderer ve Shader'a sahiptir; `ApControl` onun üzerinden çalışır, bu yüzden `OpenGL.shadeOpA()` yalnız `s.shadeOpA()` çağrısını iletir (bir tür proxy, delegation). Hocanın benzetmesi: bölüm başkanı, toplantıda olduğu için bir belgenin imzasını yetkili yardımcısına yönlendirir; iş başkasına devredilir.
  - Bu tasarımda kalıtım yoktur, yalnız sahiplik vardır; delegate metot fazlalığı ve yeni sürüm için üç sınıf gerekmesi onu tercih edilmez kılar.
- Çözüm 3'ün kod ayrıntıları (ekrandan, 02:10:00 ve 02:30:00)
  - Ara sürüm: `public ApControl( ComponentFactory library ) { this.library = library; r = library.createRenderer(); s = library.createShader(); }`.
  - Slayttaki ek notlar: çözümlerdeki soyut sınıflar yerine arayüz (interface) de kullanılabilir; tüm çözümlerde library nesnesi `ApControl` kurucusuna parametre olarak verilirse daha doğru iş yapılmış olur, çünkü seçim karmaşıklığı koddan çıkıp yapılandırma dosyasına ötelenir. Aynı şey Çözüm 2'de Renderer ve Shader (ya da `LibraryType`) parametre verilerek yapılabilir; bu açıdan çözümler eşittir.
  - Ders sonrası not: Son slayttaki `factory.r.renderOpA()` biçimi, `ComponentFactory`'nin `r` ve `s` adlı erişilebilir alanlar tutmasını gerektirir; bu, önceki slaytlardaki `createRenderer()`/`createShader()` yapısından farklıdır ve derste ayrıntılandırılmadı.
- Çıkarılan dersler ve genel tasarım kuralları (02:32:25)
  - Neyin değiştiğini bul ve sarmala: değişeni soyut bir üst sınıf ve somut varyasyonları olarak bir kalıtım hiyerarşisine koy. Böylece client onu kara kutu olarak kullanır; `ApControl` hangi grafik kütüphanesinin çalıştığını bilmeden "bana render et" der. Bir hata bulunduğunda nereye müdahale edileceği nokta atışıyla bulunur.
  - Kalıtım yerine parça-bütün ilişkisini tercih et: oyunun tamamını kütüphaneye göre türetmek, oyun mantığındaki bir hatanın her alt sınıfta tekrar düzeltilmesine yol açar.
  - Gerçeklemelere göre değil arayüzlere göre tasarla: `ApControl` fabrikadan yalnız "bana bir renderer, bir shader ver" der; OpenGL ya da DirectX'e özgü hiçbir kodu yoktur.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.